ADVANCED SQL QUERY SYSTEM

AIM:

To implement subqueries and nested queries in the ApexMart E-Commerce Database to analyze customer purchases, product pricing, and sales performance.

OBJECTIVES:

- To understand and implement single-row subqueries.
- To implement multi-row subqueries and nested queries.
- To identify products whose prices are above the average price.
- To analyze customers with maximum purchases.
- To identify high-value customers and top-selling products.
- To generate advanced business reports for decision-making.

IMPLEMENTATION:

1.Implement Subqueries and Nested Queries:

A subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

The following types of subqueries are implemented:

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

```
SQL> SELECT item_name, unit_rate FROM catalog_item WHERE unit_rate > (SELECT AVG(unit_rate) FROM catalog_item);

ITEM_NAME                           UNIT_RATE
----------------------------------- ----------
Mechanical Numpad                       420.00
Aluminum Laptop Riser                   650.00
Noise-Canceling Earbuds                 890.00

SQL> SELECT item_name, unit_rate FROM catalog_item WHERE unit_rate = (SELECT MAX(unit_rate) FROM catalog_item);

ITEM_NAME                           UNIT_RATE
----------------------------------- ----------
Noise-Canceling Earbuds                 890.00

SQL> SELECT item_name, unit_rate FROM catalog_item WHERE cat_code IN (SELECT cat_code FROM merch_category);

ITEM_NAME                           UNIT_RATE
----------------------------------- ----------
Ergonomic Desk Mat                      280.00
Stainless Thermal Tumbler               340.00
Mechanical Numpad                       420.00
Braided Type-C Cable                    120.00
Ceramic Aroma Diffuser                  260.00
Aluminum Laptop Riser                   650.00
Noise-Canceling Earbuds                 890.00
Compact Desk Fan                        190.00
Canvas Tech Organizer                   150.00
Artisanal Herbal Tea                    100.00

10 rows selected.

SQL> SELECT item_name, unit_rate FROM catalog_item WHERE unit_rate > (SELECT AVG(unit_rate) FROM catalog_item WHERE cat_code IN (SELECT cat_code FROM merch_category));

ITEM_NAME                           UNIT_RATE
----------------------------------- ----------
Mechanical Numpad                       420.00
Aluminum Laptop Riser                   650.00
Noise-Canceling Earbuds                 890.00
```

2.Find Products Above Average Price:

This query is used to identify premium products in the ApexMart database.

The following details are retrieved:

- Products priced above the average product price.

```
SQL> SELECT item_code, item_name, unit_rate FROM catalog_item WHERE unit_rate > (SELECT AVG(unit_rate) FROM catalog_item);

ITEM_CODE  ITEM_NAME                                UNIT_RATE
---------- ---------------------------------------- ----------
ITM-103    Mechanical Numpad                           420.00
ITM-106    Aluminum Laptop Riser                       650.00
ITM-107    Noise-Canceling Earbuds                     890.00
```

- Product name, category, and price.

```
SQL> SELECT p.item_name, c.cat_title, p.unit_rate
  2  FROM catalog_item p JOIN merch_category c ON p.cat_code = c.cat_code
  3  WHERE p.unit_rate > (SELECT AVG(unit_rate) FROM catalog_item);

ITEM_NAME                 CAT_TITLE                   UNIT_RATE
------------------------- --------------------------- ---------
Mechanical Numpad         Computing & Peripherals          420.00
Aluminum Laptop Riser     Ergonomic Workstation            650.00
Noise-Canceling Earbuds   Audio & Sound                    890.00
```

- Expensive products in each category.

```
SQL> SELECT p.item_name, p.cat_code, p.unit_rate
  2  FROM catalog_item p
  3  WHERE p.unit_rate = (SELECT MAX(p2.unit_rate) FROM catalog_item p2 WHERE p2.cat_code = p.cat_code);

ITEM_NAME                 CAT_CODE  UNIT_RATE
------------------------- --------- ---------
Mechanical Numpad         CAT-101      420.00
Aluminum Laptop Riser     CAT-102      650.00
Noise-Canceling Earbuds   CAT-103      890.00
Stainless Thermal Tumbler CAT-104      340.00
Canvas Tech Organizer     CAT-105      150.00

5 rows selected.
```

- Products with prices higher than a selected category average.

```
SQL> SELECT p.item_name, c.cat_title, p.unit_rate
  2  FROM catalog_item p JOIN merch_category c ON p.cat_code = c.cat_code
  3  WHERE p.unit_rate > (SELECT AVG(p2.unit_rate) FROM catalog_item p2 WHERE p2.cat_code = p.cat_code);

ITEM_NAME                 CAT_TITLE                   UNIT_RATE
------------------------- --------------------------- ---------
Mechanical Numpad         Computing & Peripherals          420.00
Aluminum Laptop Riser     Ergonomic Workstation            650.00
Stainless Thermal Tumbler Wellness & Living                340.00
```

3.Identify Customers with Maximum Purchases:

Customer purchasing behavior is analyzed using SQL queries.

The following details are identified:

- Customer who spent the highest amount.

```
SQL> SELECT s.shopper_id, s.shopper_name, SUM(o.order_total) AS total_spent
  2  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  3  GROUP BY s.shopper_id, s.shopper_name
  4  HAVING SUM(o.order_total) = (SELECT MAX(total_sales) FROM (SELECT SUM(order_total) AS total_sales FROM sales_order GROUP BY shopper_id));

SHOPPER_ID SHOPPER_NAME                  TOTAL_SPENT
---------- ------------------------------ -----------
SHP-206    Ananya Iyer                         890.00
```

- Customers with the maximum number of order.

```
SQL> SELECT s.shopper_id, s.shopper_name, COUNT(o.order_num) AS total_orders
  2  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  3  GROUP BY s.shopper_id, s.shopper_name
  4  HAVING COUNT(o.order_num) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM sales_order GROUP BY shopper_id));

SHOPPER_ID SHOPPER_NAME                  TOTAL_ORDERS
---------- ------------------------------ ------------
SHP-201    Aarav Sharma                              1
SHP-202    Meera Krishnan                            1
SHP-203    Rohan Desai                               1
SHP-204    Pooja Nair                                1
SHP-205    Vikram Sengupta                           1
SHP-206    Ananya Iyer                               1
SHP-207    Karthik Raman                             1
SHP-208    Divya Nambiar                             1
SHP-209    Siddharth Verma                           1
SHP-210    Tanvi Kulkarni                            1

10 rows selected.
```

• Customers whose spending is above average.

```
SQL> SELECT s.shopper_id, s.shopper_name, SUM(o.order_total) AS total_spent
  2  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  3  GROUP BY s.shopper_id, s.shopper_name
  4  HAVING SUM(o.order_total) > (SELECT AVG(shopper_total) FROM (SELECT SUM(order_total) AS shopper_total FROM sales_order GROUP BY shopper_id));

SHOPPER_ID SHOPPER_NAME                   TOTAL_SPENT
---------- ------------------------------ -----------
SHP-206    Ananya Iyer                         890.00
SHP-203    Rohan Desai                         650.00
SHP-201    Aarav Sharma                        560.00
SHP-210    Tanvi Kulkarni                      560.00
```

• Top 5 valuable customers.

```
SQL> SELECT * FROM (
  2    SELECT s.shopper_id, s.shopper_name, SUM(o.order_total) AS total_spent
  3    FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  4    GROUP BY s.shopper_id, s.shopper_name
  5    ORDER BY total_spent DESC
  6  ) WHERE ROWNUM <= 5;

SHOPPER_ID SHOPPER_NAME                   TOTAL_SPENT
---------- ------------------------------ -----------
SHP-206    Ananya Iyer                         890.00
SHP-203    Rohan Desai                         650.00
SHP-201    Aarav Sharma                        560.00
SHP-210    Tanvi Kulkarni                      560.00
SHP-207    Karthik Raman                       460.00
```

4.Generate Complex Business Queries:

Advanced SQL queries are created for business analysis.

Query 1: Best-Selling Product

Identifies products with maximum sales quantity and displays product name, total quantity sold, and total revenue generated.

```
SQL> SELECT p.item_name, SUM(od.item_qty) AS total_quantity_sold, SUM(od.item_qty * od.unit_rate) AS total_revenue
  2  FROM catalog_item p JOIN order_line_item od ON p.item_code = od.item_code
  3  GROUP BY p.item_code, p.item_name
  4  HAVING SUM(od.item_qty) = (SELECT MAX(total_qty) FROM (SELECT SUM(item_qty) AS total_qty FROM order_line_item GROUP BY item_code));

ITEM_NAME                     TOTAL_QUANTITY_SOLD TOTAL_REVENUE
----------------------------- ------------------- -------------
Ergonomic Desk Mat                              5       1350.00
```

Query 2: High-Value Customers

Identifies customers whose total purchase amount is greater than the average customer spending.

```
SQL> SELECT s.shopper_id, s.shopper_name, SUM(o.order_total) AS total_spending
  2  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  3  GROUP BY s.shopper_id, s.shopper_name
  4  HAVING SUM(o.order_total) > (SELECT AVG(shopper_spending) FROM (SELECT SUM(order_total) AS shopper_spending FROM sales_order GROUP BY shopper_id));

SHOPPER_ID SHOPPER_NAME                   TOTAL_SPENDING
---------- ------------------------------ --------------
SHP-206    Ananya Iyer                            890.00
SHP-203    Rohan Desai                            650.00
SHP-201    Aarav Sharma                           560.00
SHP-210    Tanvi Kulkarni                         560.00
```

Query 3: Category Performance Analysis

Analyzes category-wise sales performance by identifying the highest revenue-generating category, number of products sold in each category, and average product price by category.

```
SQL> SELECT * FROM (
  2    SELECT c.cat_title, NVL(SUM(od.item_qty * od.unit_rate), 0) AS total_revenue
  3    FROM merch_category c
  4    LEFT JOIN catalog_item p ON c.cat_code = p.cat_code
  5    LEFT JOIN order_line_item od ON p.item_code = od.item_code
  6    GROUP BY c.cat_code, c.cat_title
  7    ORDER BY total_revenue DESC
  8  ) WHERE ROWNUM = 1;

CAT_TITLE                     TOTAL_REVENUE
----------------------------- -------------
Ergonomic Workstation                2190.00
```

```
SQL> SELECT c.cat_title, NVL(SUM(od.item_qty), 0) AS products_sold
  2  FROM merch_category c
  3  LEFT JOIN catalog_item p ON c.cat_code = p.cat_code
  4  LEFT JOIN order_line_item od ON p.item_code = od.item_code
  5  GROUP BY c.cat_code, c.cat_title;

CAT_TITLE                      PRODUCTS_SOLD
------------------------------ -------------
Ergonomic Workstation                      7
Computing & Peripherals                    4
Wellness & Living                          2
Audio & Sound                              1
Travel & Accessories                       1

5 rows selected.
```

```
SQL> SELECT c.cat_title, AVG(p.unit_rate) AS average_product_price
  2  FROM merch_category c JOIN catalog_item p ON c.cat_code = p.cat_code
  3  GROUP BY c.cat_code, c.cat_title;

CAT_TITLE                      AVERAGE_PRODUCT_PRICE
------------------------------ ---------------------
Audio & Sound                                890.00
Ergonomic Workstation                        373.33
Computing & Peripherals                      270.00
Wellness & Living                            233.33
Travel & Accessories                         150.00

5 rows selected.
```

Query 4: Customer Purchase History

Displays customer name, number of orders, total purchase amount, and most purchased product.

```
SQL> WITH order_totals AS (
  2    SELECT shopper_id, COUNT(order_num) AS number_of_orders, SUM(order_total) AS total_purchase_amount
  3    FROM sales_order GROUP BY shopper_id
  4  ), product_totals AS (
  5    SELECT o.shopper_id, p.item_name, SUM(od.item_qty) AS total_quantity,
  6    ROW_NUMBER() OVER (PARTITION BY o.shopper_id ORDER BY SUM(od.item_qty) DESC, p.item_name) AS rn
  7    FROM sales_order o JOIN order_line_item od ON o.order_num = od.order_num
  8    JOIN catalog_item p ON od.item_code = p.item_code GROUP BY o.shopper_id, p.item_code, p.item_name
  9  )
 10  SELECT s.shopper_name, NVL(ot.number_of_orders, 0) AS number_of_orders,
 11  NVL(ot.total_purchase_amount, 0) AS total_purchase_amount, pt.item_name AS most_purchased_product
 12  FROM shopper_account s LEFT JOIN order_totals ot ON s.shopper_id = ot.shopper_id
 13  LEFT JOIN product_totals pt ON s.shopper_id = pt.shopper_id AND pt.rn = 1;

SHOPPER_NAME                   NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
------------------------------ ---------------- --------------------- ----------------------
Aarav Sharma                                  1                   560 Ergonomic Desk Mat
Meera Krishnan                                1                   260 Ceramic Aroma Diffuser
Rohan Desai                                   1                   650 Aluminum Laptop Riser
Pooja Nair                                    1                   420 Mechanical Numpad
Vikram Sengupta                               1                   190 Compact Desk Fan
Ananya Iyer                                   1                   890 Noise-Canceling Earbuds
Karthik Raman                                 1                   460 Stainless Thermal Tumbler
Divya Nambiar                                 1                   380 Canvas Tech Organizer
Siddharth Verma                               1                   240 Braided Type-C Cable
Tanvi Kulkarni                                1                   560 Ergonomic Desk Mat

10 rows selected.
```

5. ADVANCED SQL REPORTS:

Report 1: Premium Product Report

This report displays product name, category, price, and stock availability condition for products whose prices are above the average product price.

```
SQL> SELECT p.item_name, c.cat_title, p.unit_rate,
  2  CASE WHEN p.available_stock > 0 THEN 'Available' ELSE 'Out of Stock' END AS stock_availability
  3  FROM catalog_item p JOIN merch_category c ON p.cat_code = c.cat_code
  4  WHERE p.unit_rate > (SELECT AVG(unit_rate) FROM catalog_item);

ITEM_NAME                 CAT_TITLE                      UNIT_RATE STOCK_AVAILABILITY
------------------------- ------------------------------ --------- -------------------
Mechanical Numpad         Computing & Peripherals           420.00 Available
Aluminum Laptop Riser     Ergonomic Workstation             650.00 Available
Noise-Canceling Earbuds   Audio & Sound                     890.00 Available
```

Report 2: Customer Value Report

This report displays customer name, total orders, total spending, and customer category.

```
SQL> SELECT s.shopper_name, COUNT(o.order_num) AS total_orders, NVL(SUM(o.order_total), 0) AS total_spending,
  2  CASE WHEN NVL(SUM(o.order_total), 0) >= 600 THEN 'Premium' ELSE 'Regular' END AS customer_category
  3  FROM shopper_account s LEFT JOIN sales_order o ON s.shopper_id = o.shopper_id
  4  GROUP BY s.shopper_id, s.shopper_name;

SHOPPER_NAME                   TOTAL_ORDERS TOTAL_SPENDING CUSTOMER_CATEGORY
------------------------------ ------------ -------------- ------------------
Aarav Sharma                              1         560.00 Regular
Meera Krishnan                            1         260.00 Regular
Rohan Desai                               1         650.00 Premium
Pooja Nair                                1         420.00 Regular
Vikram Sengupta                           1         190.00 Regular
Ananya Iyer                               1         890.00 Premium
Karthik Raman                             1         460.00 Regular
Divya Nambiar                             1         380.00 Regular
Siddharth Verma                           1         240.00 Regular
Tanvi Kulkarni                            1         560.00 Regular

10 rows selected.
```

Report 3: Sales Performance Report

This report displays product-wise sales, category-wise revenue, and top-selling products.

```
SQL> SELECT p.item_name, NVL(SUM(od.item_qty), 0) AS quantity_sold, NVL(SUM(od.item_qty * od.unit_rate), 0) AS total_revenue
  2  FROM catalog_item p LEFT JOIN order_line_item od ON p.item_code = od.item_code
  3  GROUP BY p.item_code, p.item_name;

ITEM_NAME                      QUANTITY_SOLD TOTAL_REVENUE
------------------------------ ------------- -------------
Ergonomic Desk Mat                         5       1350.00
Braided Type-C Cable                       3        360.00
Stainless Thermal Tumbler                  1        340.00
Mechanical Numpad                          1        420.00
Ceramic Aroma Diffuser                     1        260.00
Aluminum Laptop Riser                      1        650.00
Noise-Canceling Earbuds                    1        890.00
Compact Desk Fan                           1        190.00
Canvas Tech Organizer                      1        150.00
Artisanal Herbal Tea                       0          0.00

10 rows selected.
```

```
SQL> SELECT c.cat_title, NVL(SUM(od.item_qty * od.unit_rate), 0) AS total_revenue
  2  FROM merch_category c LEFT JOIN catalog_item p ON c.cat_code = p.cat_code
  3  LEFT JOIN order_line_item od ON p.item_code = od.item_code
  4  GROUP BY c.cat_code, c.cat_title;

CAT_TITLE                      TOTAL_REVENUE
------------------------------ -------------
Ergonomic Workstation                2190.00
Audio & Sound                         890.00
Computing & Peripherals               780.00
Wellness & Living                     600.00
Travel & Accessories                  150.00

5 rows selected.
```

```
SQL> SELECT * FROM (
  2    SELECT p.item_name, SUM(od.item_qty) AS quantity_sold
  3    FROM catalog_item p JOIN order_line_item od ON p.item_code = od.item_code
  4    GROUP BY p.item_code, p.item_name ORDER BY quantity_sold DESC
  5  ) WHERE ROWNUM <= 5;

ITEM_NAME                      QUANTITY_SOLD
------------------------------ -------------
Ergonomic Desk Mat                         5
Braided Type-C Cable                       3
Aluminum Laptop Riser                      1
Mechanical Numpad                          1
Noise-Canceling Earbuds                    1
```

Report 4: Business Decision Report

This report identifies:

- High-performing products.

```
SQL> SELECT p.item_name, SUM(od.item_qty) AS quantity_sold, SUM(od.item_qty * od.unit_rate) AS total_revenue
  2  FROM catalog_item p JOIN order_line_item od ON p.item_code = od.item_code
  3  GROUP BY p.item_code, p.item_name
  4  HAVING SUM(od.item_qty) >= (SELECT AVG(item_qty) FROM (SELECT SUM(item_qty) AS item_qty FROM order_line_item GROUP BY item_code))
  5  ORDER BY total_revenue DESC;

ITEM_NAME                      QUANTITY_SOLD TOTAL_REVENUE
------------------------------ ------------- -------------
Ergonomic Desk Mat                         5       1350.00
Braided Type-C Cable                       3        360.00
```

- Low-performing products.

```
SQL> SELECT p.item_name, NVL(SUM(od.item_qty), 0) AS quantity_sold, NVL(SUM(od.item_qty * od.unit_rate), 0) AS total_revenue
  2  FROM catalog_item p LEFT JOIN order_line_item od ON p.item_code = od.item_code
  3  GROUP BY p.item_code, p.item_name
  4  HAVING NVL(SUM(od.item_qty), 0) < (SELECT AVG(item_qty) FROM (SELECT SUM(item_qty) AS item_qty FROM order_line_item GROUP BY item_code));

ITEM_NAME                      QUANTITY_SOLD TOTAL_REVENUE
------------------------------ ------------- -------------
Stainless Thermal Tumbler                  1        340.00
Mechanical Numpad                          1        420.00
Ceramic Aroma Diffuser                     1        260.00
Aluminum Laptop Riser                      1        650.00
Noise-Canceling Earbuds                    1        890.00
Compact Desk Fan                           1        190.00
Canvas Tech Organizer                      1        150.00
Artisanal Herbal Tea                       0          0.00

8 rows selected.
```

- High-value customers.

```
SQL> SELECT s.shopper_name, SUM(o.order_total) AS total_spending
  2  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  3  GROUP BY s.shopper_id, s.shopper_name
  4  HAVING SUM(o.order_total) > (SELECT AVG(customer_spending) FROM (SELECT SUM(order_total) AS customer_spending FROM sales_order GROUP BY shopper_id))
  5  ORDER BY total_spending DESC;

SHOPPER_NAME                   TOTAL_SPENDING
------------------------------ --------------
Ananya Iyer                            890.00
Rohan Desai                            650.00
Aarav Sharma                           560.00
Tanvi Kulkarni                         560.00
```

• Revenue trends.

```
SQL> SELECT TO_CHAR(TO_DATE(booking_date, 'DD-MON-YY'), 'YYYY-MM') AS sales_month, COUNT(order_num) AS total_orders, SUM(order_total) AS monthly_revenue
  2  FROM sales_order
  3  GROUP BY TO_CHAR(TO_DATE(booking_date, 'DD-MON-YY'), 'YYYY-MM')
  4  ORDER BY sales_month;

SALES_MONTH TOTAL_ORDERS MONTHLY_REVENUE
----------- ------------ ---------------
2026-09              10         4610.00
```